

LAB ACTION PLAN FOR WEEK 4

<https://github.com/DiamondDevil-dotcom/mavenweb.git>

<https://github.com/DiamondDevil-dotcom/maven-calculator.git>

Objective:

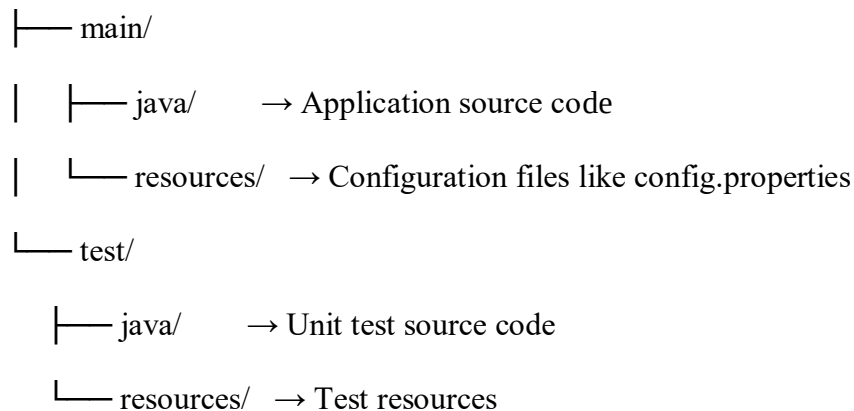
To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications. src/



The compiled files and reports are generated in the **target/** directory after a successful build.

2. Steps to Perform Maven Build and Testing

----mvn clean install

- **clean:** Deletes the previous build (**target/** folder)

- **install**: Builds, tests, and installs the package to local **.m2** repository

After execution:

- **target/** folder contains compiled **.class** files and the final **.jar** or **.war**
- Artifact is stored in:
 `~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In **pom.xml**:

```
<dependency>

    <groupId>com.google.code.gson</groupId>

    <artifactId>gson</artifactId>

    <version>2.10</version>

</dependency>
```

After running:

```
mvn clean install
```

Check:

- Gson JAR is downloaded to `.m2/repository/com/google/code/gson/gson/`
- If version is wrong or not found → **BUILD FAILURE** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└─ 1.1. Launch Eclipse workspace



Step 2. Install Maven Plugin (if not installed)

- └─ 2.1. Go to "Help" in the top menu
 - └─ 2.1.1. Click "Eclipse Marketplace"
 - └─ 2.1.2. Search for "Maven Integration for Eclipse"
 - └─ 2.1.3. Install the plugin if not already installed

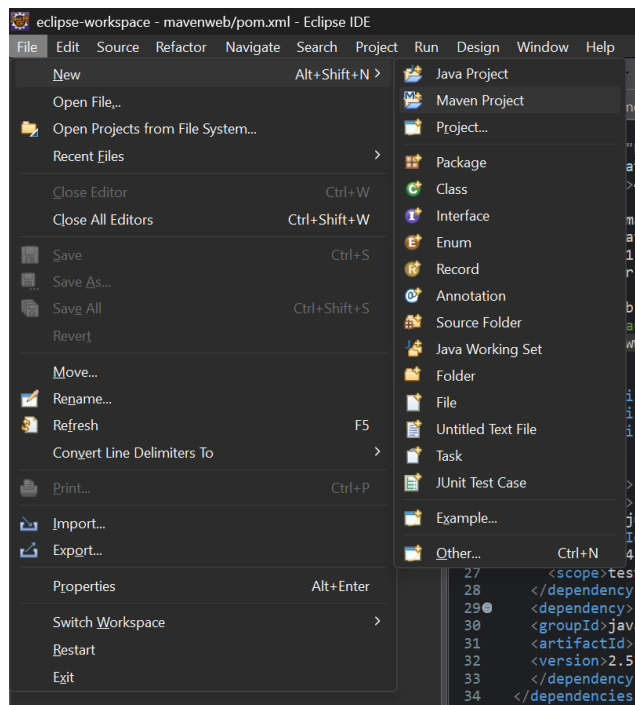


Step 3. Create a New Maven Project

└─ 3.1. File -> New -> Project...

└─ 3.1.1. Expand "Maven"

└─ 3.1.2. Select "Maven Project" and click "Next"

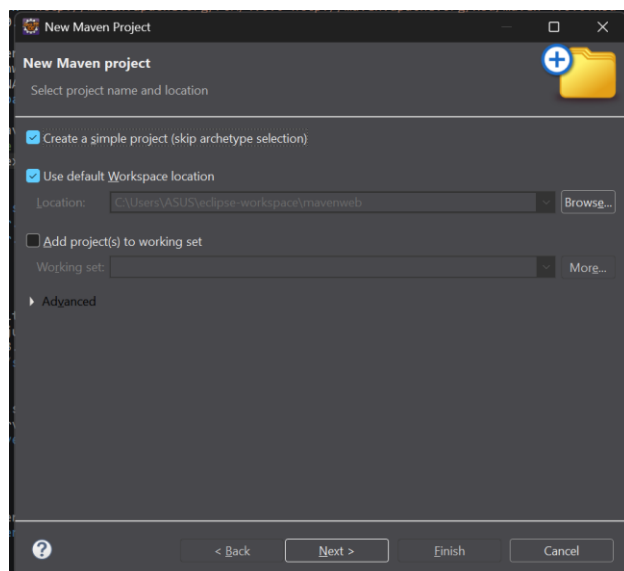


Step 4. Set Project Configuration

- └─ 4.1. Select workspace location (default or custom)
- └─ 4.2. Click "Next"

Step 5. Choose Maven Archetype

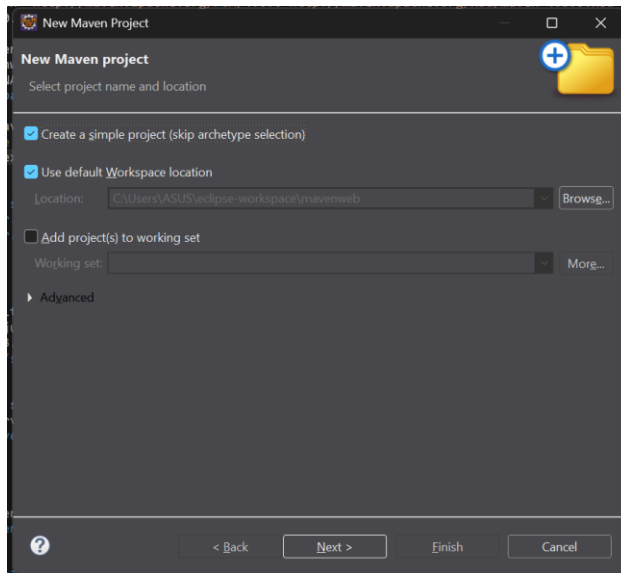
- └─ 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")
- └─ 5.2. Click "Next"



Step 6. Define Project Metadata

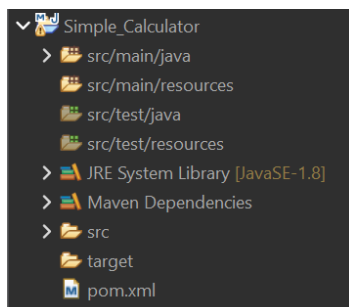
- └─ 6.1. Group ID: (e.g., com.example)
- └─ 6.2. Artifact ID: (e.g., my-maven-project)
- └─ 6.3. Version: (default is usually fine)
- └─ 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.



Step 7. Maven Project Created

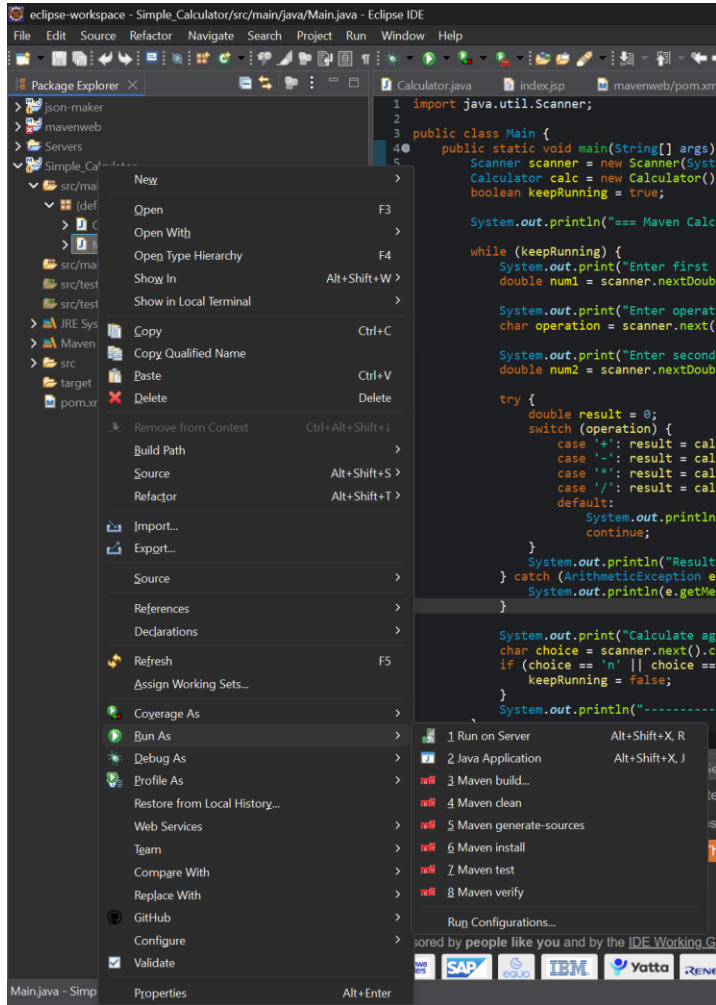
- └─ 7.1. Project structure is generated with a standard Maven layout
- └─ 7.2. Includes:
 - └─ src/main/java (for Java source code)
 - └─ src/test/java (for test code)
 - └─ pom.xml (Maven configuration file)



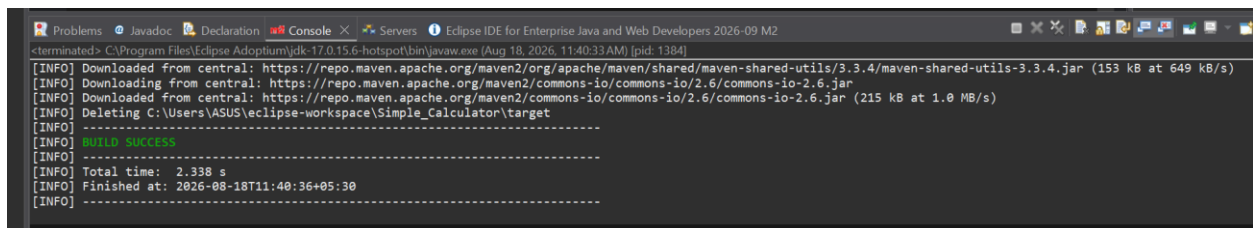
Step 8. Update Project Settings (if needed)

- └─ 8.1. Right-click on the project -> Maven -> Update Project...
- └─ 8.2. Ensure dependencies are up to date

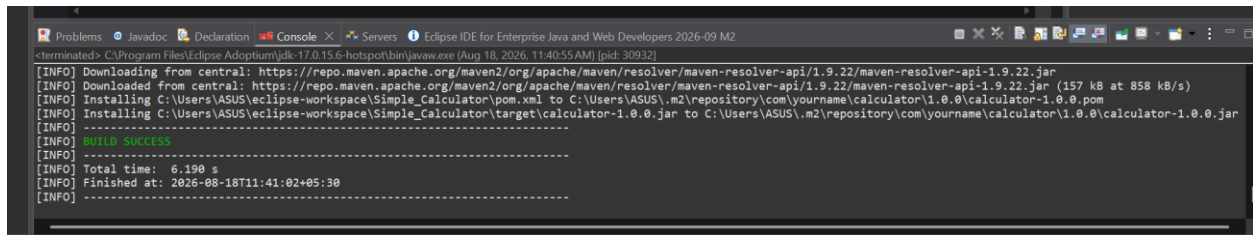
Step 9. Build and Run Maven Project



- └─ 9.1. Right-click on App.java -> Run As -> Maven Clean

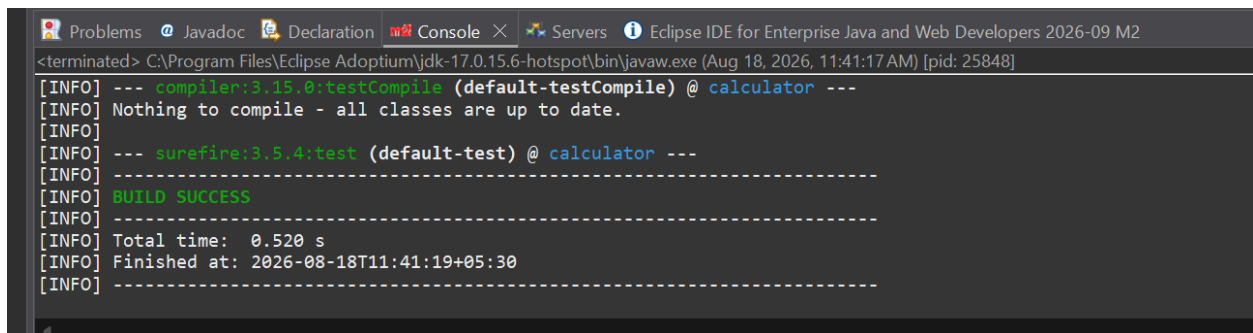


9.1.1. Right-click on App.java -> Run As -> Maven Install



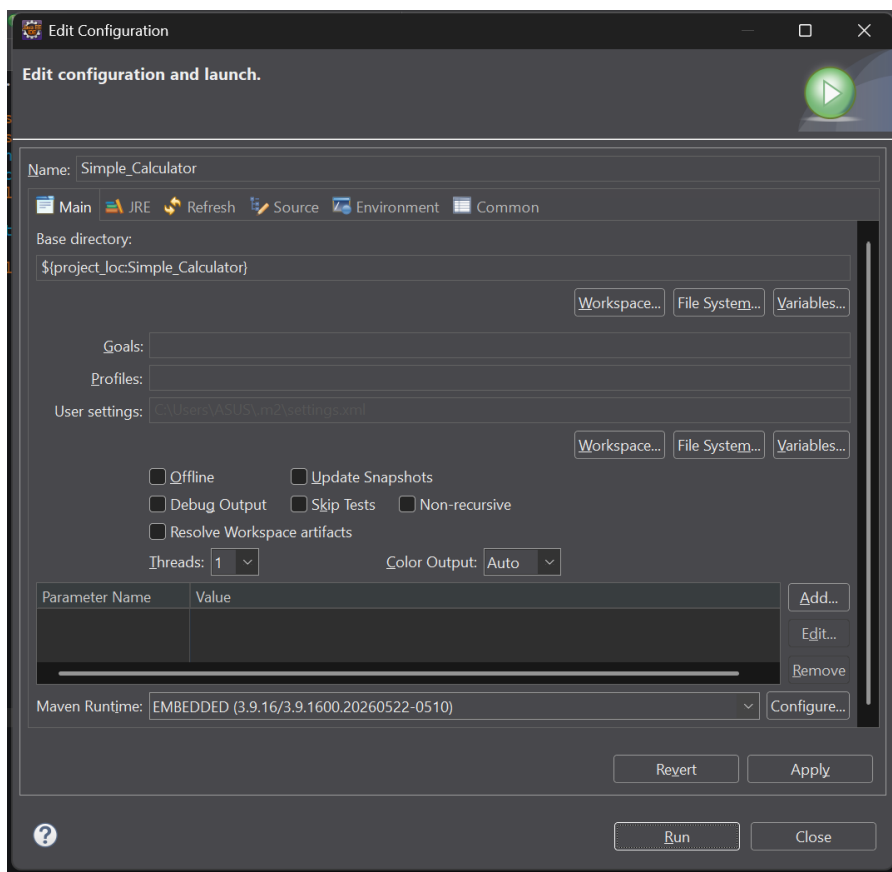
```
terminated> C:\Program Files\Eclipse Adoptium\jdk-17.0.15-hotspot\bin\javaw.exe (Aug 18, 2026, 11:40:55 AM) [pid: 30932]
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.22/maven-resolver-api-1.9.22.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.22/maven-resolver-api-1.9.22.jar (157 kB at 858 kB/s)
[INFO] Installing C:\Users\ASUS\workspace\Simple_Calculator\pom.xml to C:\Users\ASUS\.m2\repository\com\yourname\calculator\1.0.0\calculator-1.0.0.pom
[INFO] Installing C:\Users\ASUS\workspace\Simple_Calculator\target\calculator-1.0.0.jar to C:\Users\ASUS\.m2\repository\com\yourname\calculator\1.0.0\calculator-1.0.0.jar
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.190 s
[INFO] Finished at: 2026-08-18T11:41:02+05:30
[INFO] -----
```

9.1.2. Right-click on App.java -> Run As -> Maven Test



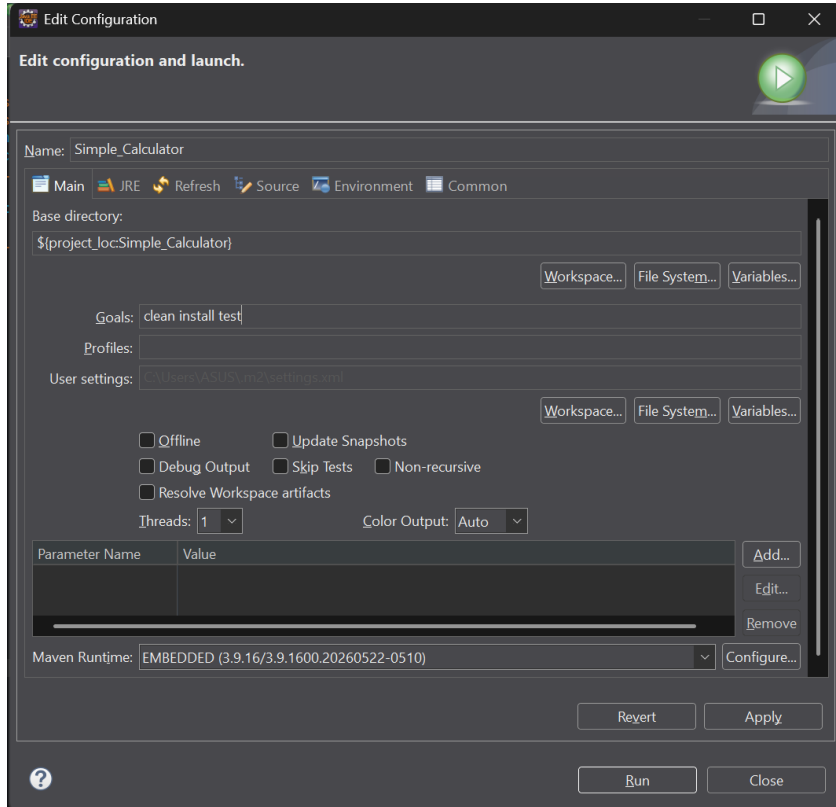
```
terminated> C:\Program Files\Eclipse Adoptium\jdk-17.0.15-hotspot\bin\javaw.exe (Aug 18, 2026, 11:41:17 AM) [pid: 25848]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ calculator ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.5.4:test (default-test) @ calculator ---
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.520 s
[INFO] Finished at: 2026-08-18T11:41:19+05:30
[INFO] -----
```

9.1.3. Right-click on App.java -> Run As -> Maven Build

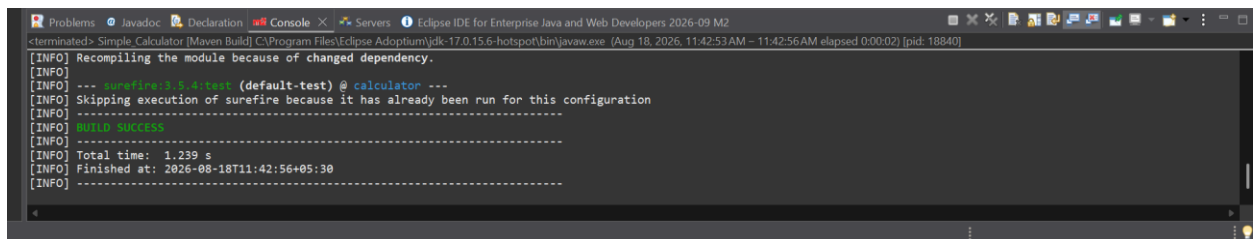


Step 10. In the Maven Build dialog:

- └ Enter Goals: clean install test
- └ Click on Apply -> Click on Run

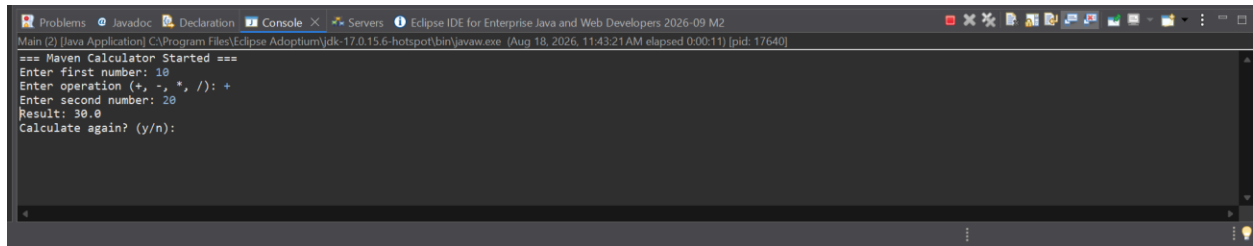


Step 11. Check console for BUILD SUCCESS message.



Step 12. Run the application:

- └ Right-click on App.java -> Run As -> Java Application
- └ Output: "Hello World" displayed.



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the execution of a Java application titled 'Main (2) [Java Application]'. The application is a simple calculator that prompts the user for a first number (10), an operation (+), and a second number (20). It then calculates the result (30.0) and asks if the user wants to calculate again. The console also shows the path to the Java executable and the elapsed time of 0:00:11.

```
Main (2) [Java Application] C:\Program Files\Eclipse Adoptium\jdk-17.0.15-hotspot\bin\javaw.exe (Aug 18, 2026, 11:43:21AM elapsed 0:00:11) [pid: 17640]
=== Maven Calculator Started ===
Enter first number: 10
Enter operation (+, -, *, /): +
Enter second number: 20
Result: 30.0
Calculate again? (y/n):
```

Creation of Maven web Java Project

Step 1: Open Eclipse

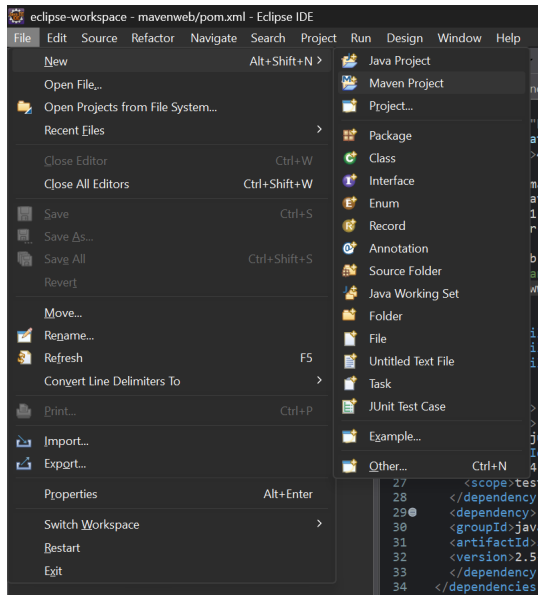
└─ 1.1 Launch Eclipse IDE.



└─ 1.2 Select or create a workspace.

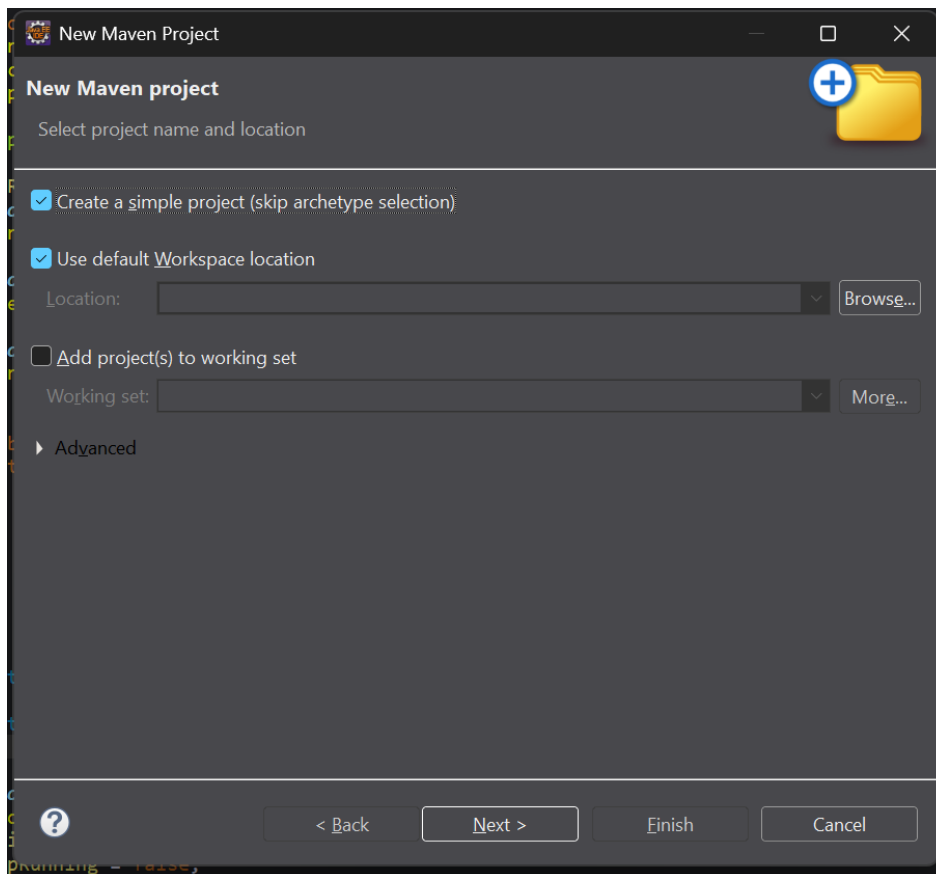
Step 2: Create a New Maven Project

└─ 2.1. File -> New -> Project...



└─ 2.1.1. Expand "Maven"

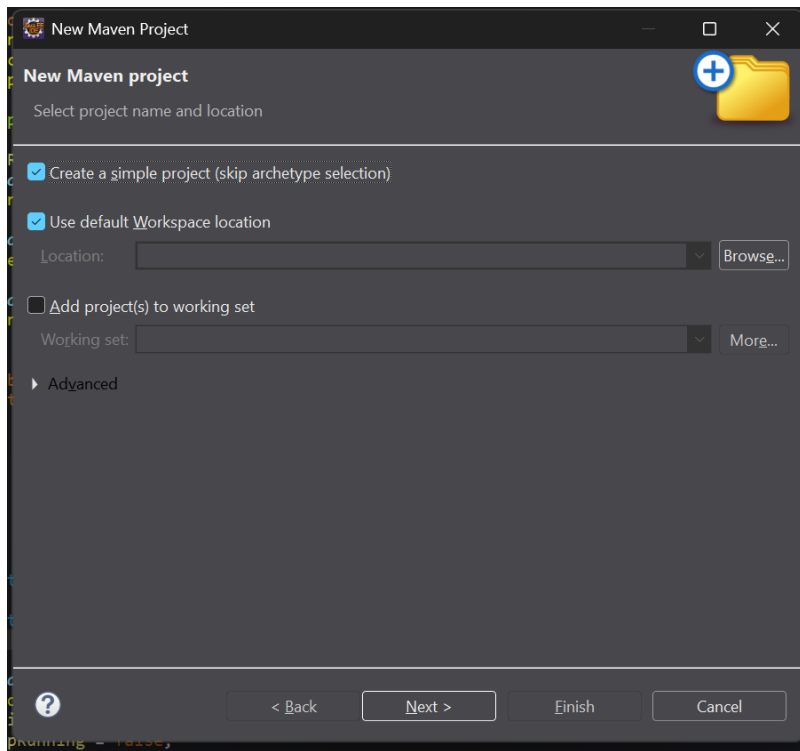
└─ 2.1.2. Select "Maven Project" and click "Next"



Step 3: Choose Maven Archetype

└─ 3.1. Select an archetype(e.g. "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")

└─ 3.2. Click "Next"



Step 4: Configure the Maven Project

└─ 4.1 Group Id: Enter a group ID (e.g., com.example).

└─ 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

└─ 4.3 Click **Finish** to create the project.

New Maven Project

Configure project

Artifact

Group Id:

Artifact Id:

Version:

Packaging:

Name:

Description:

Parent Project

Group Id:

Artifact Id:

Version:

Advanced

Step 5: Add Maven Dependencies

- 5.1 Open the ****pom.xml**** file in the Maven project.
- 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

```

1  http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
2  1 <?xml version="1.0" encoding="UTF-8"?>
3  2
4  3 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5  4 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
6  5 <modelVersion>4.0.0</modelVersion>
7  6
8  7 <groupId>com.mavenweb</groupId>
9  8 <artifactId>mavenweb</artifactId>
10 9 <version>0.0.1-SNAPSHOT</version>
11 10 <packaging>war</packaging>
12 11
13 12 <name>mavenweb Maven Webapp</name>
14 13 <!-- FIXME change it to the project's website -->
15 14 <url>http://www.example.com</url>
16 15
17 16 <properties>
18 17 <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
19 18 <maven.compiler.source>8</maven.compiler.source>
20 19 <maven.compiler.target>8</maven.compiler.target>
21 20 </properties>
22 21
23 22 <dependencies>
24 23 <dependency>
25 24 <groupId>junit</groupId>
26 25 <artifactId>junit</artifactId>
27 26 <version>4.13.1</version>
28 27 <scope>test</scope>
29 28 </dependency>
30 29 <dependency>
31 30 <groupId>javax.servlet</groupId>
32 31 <artifactId>servlet-api</artifactId>
33 32 <version>2.5</version>
34 33 </dependency>
35 34 </dependencies>
36 35
37 36 <build>
38 37 <finalName>mavenweb</finalName>
39 38 <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
40 39 <plugins>
41 40 <plugin>

```

```

33     </dependency>
34 </dependencies>
35
36 <build>
37     <finalName>mavenweb</finalName>
38     <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
39     <plugins>
40     <plugin>
41         <artifactId>maven-clean-plugin</artifactId>
42         <version>3.4.0</version>
43     </plugin>
44     <!-- see http://maven.apache.org/ref/current/maven-core/default-bindings.html#Plugin_bindings_for_war_packaging -->
45     <plugin>
46         <artifactId>maven-resources-plugin</artifactId>
47         <version>3.3.1</version>
48     </plugin>
49     <plugin>
50         <artifactId>maven-compiler-plugin</artifactId>
51         <version>3.13.0</version>
52     </plugin>
53     <plugin>
54         <artifactId>maven-surefire-plugin</artifactId>
55         <version>3.3.0</version>
56     </plugin>
57     <plugin>
58         <artifactId>maven-war-plugin</artifactId>
59         <version>3.4.0</version>
60     </plugin>
61     <plugin>
62         <artifactId>maven-install-plugin</artifactId>
63         <version>3.1.2</version>
64     </plugin>
65     <plugin>
66         <artifactId>maven-deploy-plugin</artifactId>
67         <version>3.1.2</version>
68     </plugin>
69 </plugins>
70 </pluginManagement>
71 </build>
72 </project>
73

```

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

└─ Example:

```

<<<xml

```

```

<dependency>

```

```

    <groupId>javax.servlet</groupId>

```

```

    <artifactId>javax.servlet-api</artifactId>

```

```

    <version>4.0.1</version>

```

```

    <scope>provided</scope>

```

```

</dependency>

```

```

...

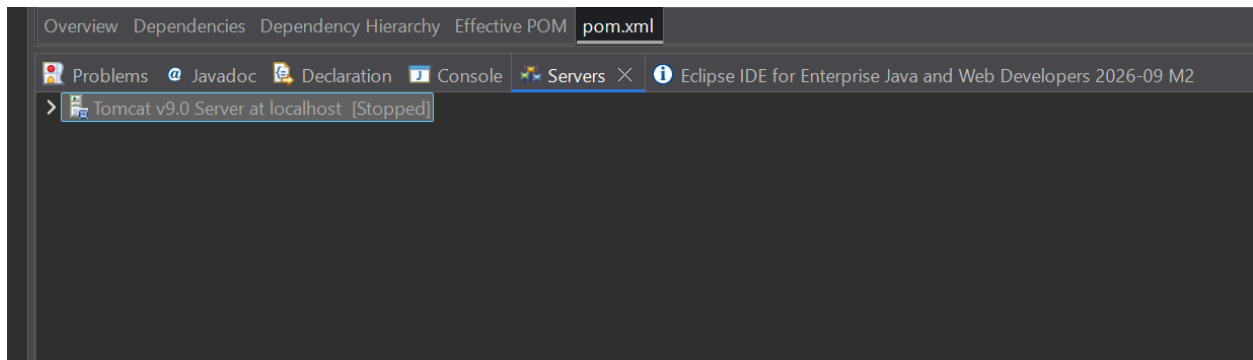
```

Step 6:- Configure server:

└─ Window -> Show View -> Servers

└─ Add server -> Select Tomcat v9.0 server -> Click Next

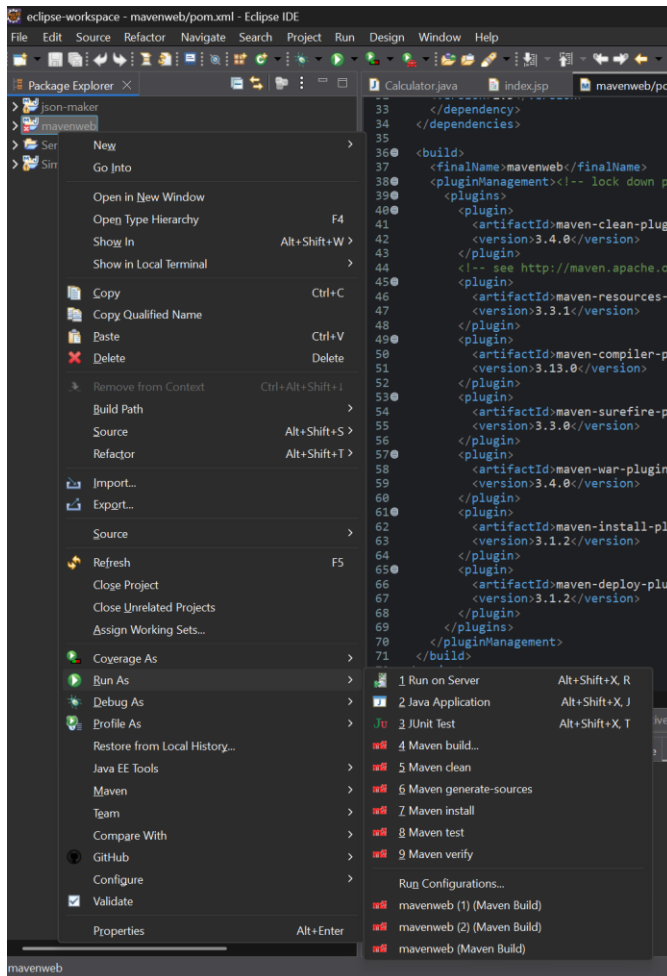
└─ Configure server options (e.g., ports, server location).



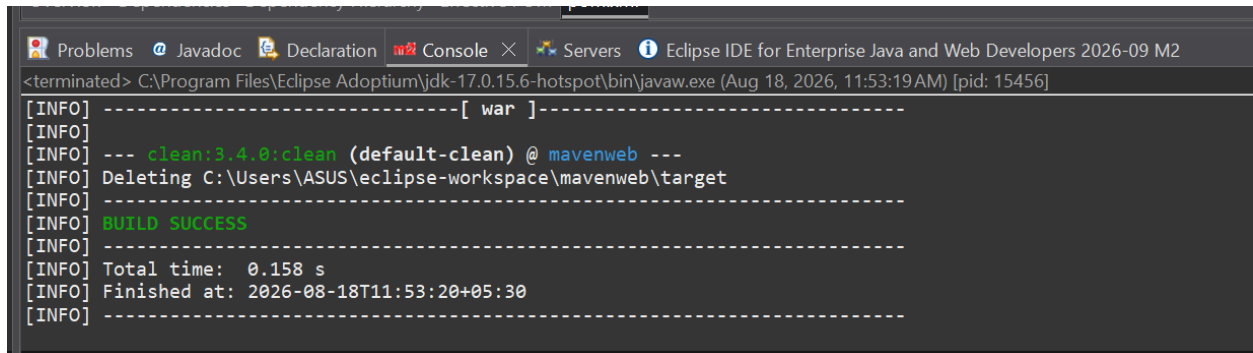
Step 7:- Modify 'tomcat-users.xml':

└─ Add role and user details under <tomcat-users> tag.

Step 8:. Build the project:



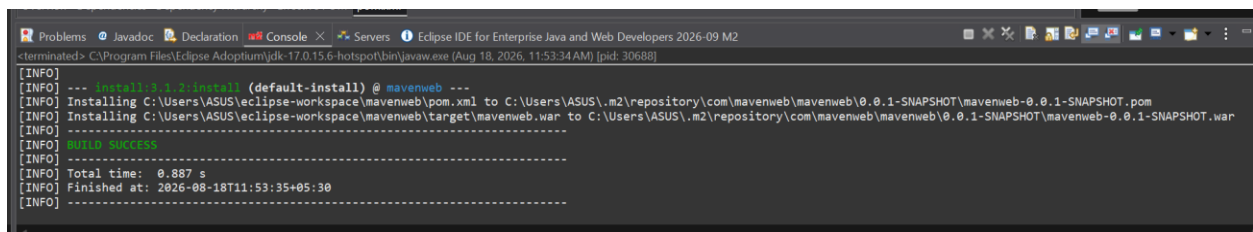
└─ Right-click on index.jsp -> Run As -> Maven Clean



The screenshot shows the Eclipse IDE console with the 'Console' tab selected. The output displays the execution of the 'maven:clean' goal. It starts with a separator line, followed by '[INFO] --- clean:3.4.0:clean (default-clean) @ mavenweb ---'. The next line shows 'Deleting C:\Users\ASUS\eclipse-workspace\mavenweb\target'. This is followed by another separator line, 'BUILD SUCCESS', and another separator line. The final lines show 'Total time: 0.158 s' and 'Finished at: 2026-08-18T11:53:20+05:30'.

```
<terminated> C:\Program Files\Eclipse Adoptium\jdk-17.0.15.6-hotspot\bin\javaw.exe (Aug 18, 2026, 11:53:19 AM) [pid: 15456]
[INFO] -----[ war ]-----
[INFO]
[INFO] --- clean:3.4.0:clean (default-clean) @ mavenweb ---
[INFO] Deleting C:\Users\ASUS\eclipse-workspace\mavenweb\target
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.158 s
[INFO] Finished at: 2026-08-18T11:53:20+05:30
[INFO] -----
```

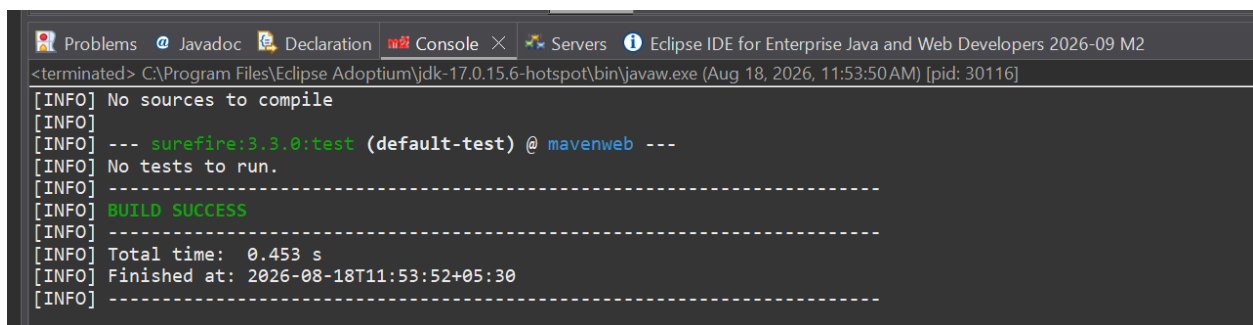
└─ Right-click on index.jsp -> Run As -> Maven Install



The screenshot shows the Eclipse IDE console with the 'Console' tab selected. The output displays the execution of the 'maven:install' goal. It starts with a separator line, followed by '[INFO] --- install:3.0.0:install (default-install) @ mavenweb ---'. The next line shows 'Installing C:\Users\ASUS\eclipse-workspace\mavenweb\pom.xml to C:\Users\ASUS\.m2\repository\com\mavenweb\mavenweb-0.0.1-SNAPSHOT\mavenweb-0.0.1-SNAPSHOT.pom'. The following line shows 'Installing C:\Users\ASUS\eclipse-workspace\mavenweb\target\mavenweb.war to C:\Users\ASUS\.m2\repository\com\mavenweb\mavenweb-0.0.1-SNAPSHOT\mavenweb-0.0.1-SNAPSHOT.war'. This is followed by another separator line, 'BUILD SUCCESS', and another separator line. The final lines show 'Total time: 0.887 s' and 'Finished at: 2026-08-18T11:53:35+05:30'.

```
<terminated> C:\Program Files\Eclipse Adoptium\jdk-17.0.15.6-hotspot\bin\javaw.exe (Aug 18, 2026, 11:53:34 AM) [pid: 30688]
[INFO] -----
[INFO] --- install:3.0.0:install (default-install) @ mavenweb ---
[INFO] Installing C:\Users\ASUS\eclipse-workspace\mavenweb\pom.xml to C:\Users\ASUS\.m2\repository\com\mavenweb\mavenweb-0.0.1-SNAPSHOT\mavenweb-0.0.1-SNAPSHOT.pom
[INFO] Installing C:\Users\ASUS\eclipse-workspace\mavenweb\target\mavenweb.war to C:\Users\ASUS\.m2\repository\com\mavenweb\mavenweb-0.0.1-SNAPSHOT\mavenweb-0.0.1-SNAPSHOT.war
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.887 s
[INFO] Finished at: 2026-08-18T11:53:35+05:30
[INFO] -----
```

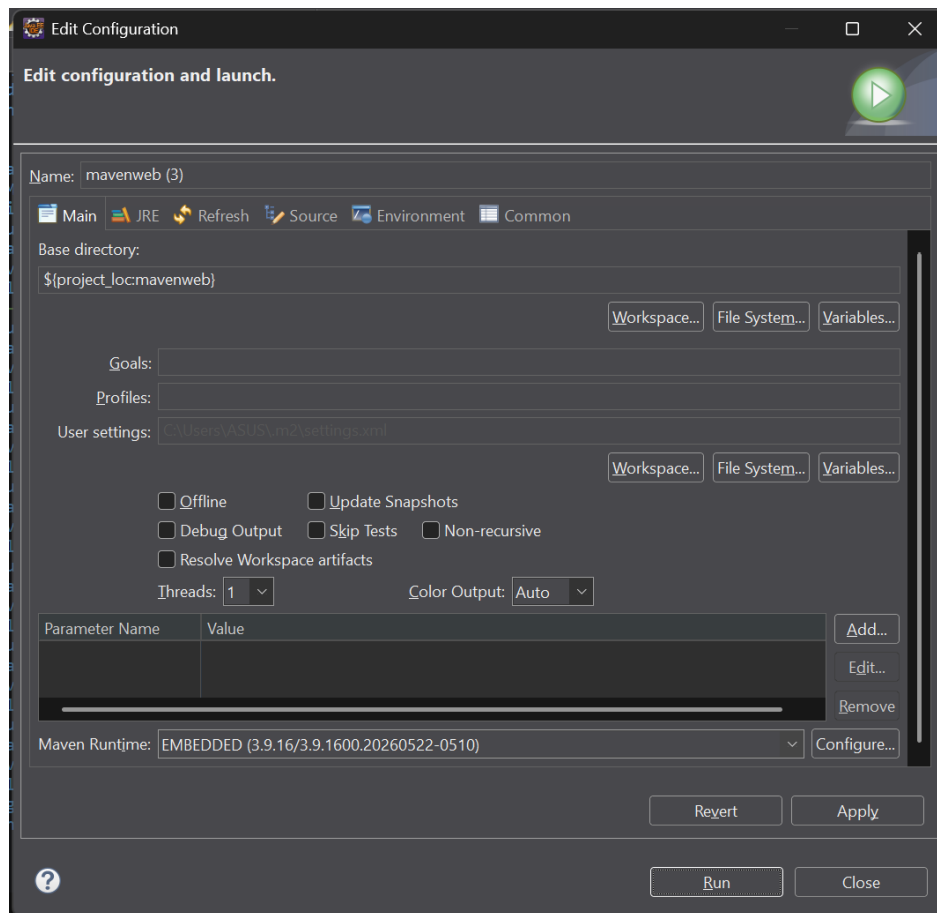
└─ Right-click on index.jsp -> Run As -> Maven Test



The screenshot shows the Eclipse IDE console with the 'Console' tab selected. The output displays the execution of the 'maven:test' goal. It starts with a separator line, followed by '[INFO] No sources to compile'. The next line shows '--- surefire:3.3.0:test (default-test) @ mavenweb ---'. The following line shows 'No tests to run.'. This is followed by another separator line, 'BUILD SUCCESS', and another separator line. The final lines show 'Total time: 0.453 s' and 'Finished at: 2026-08-18T11:53:52+05:30'.

```
<terminated> C:\Program Files\Eclipse Adoptium\jdk-17.0.15.6-hotspot\bin\javaw.exe (Aug 18, 2026, 11:53:50 AM) [pid: 30116]
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:3.3.0:test (default-test) @ mavenweb ---
[INFO] No tests to run.
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.453 s
[INFO] Finished at: 2026-08-18T11:53:52+05:30
[INFO] -----
```

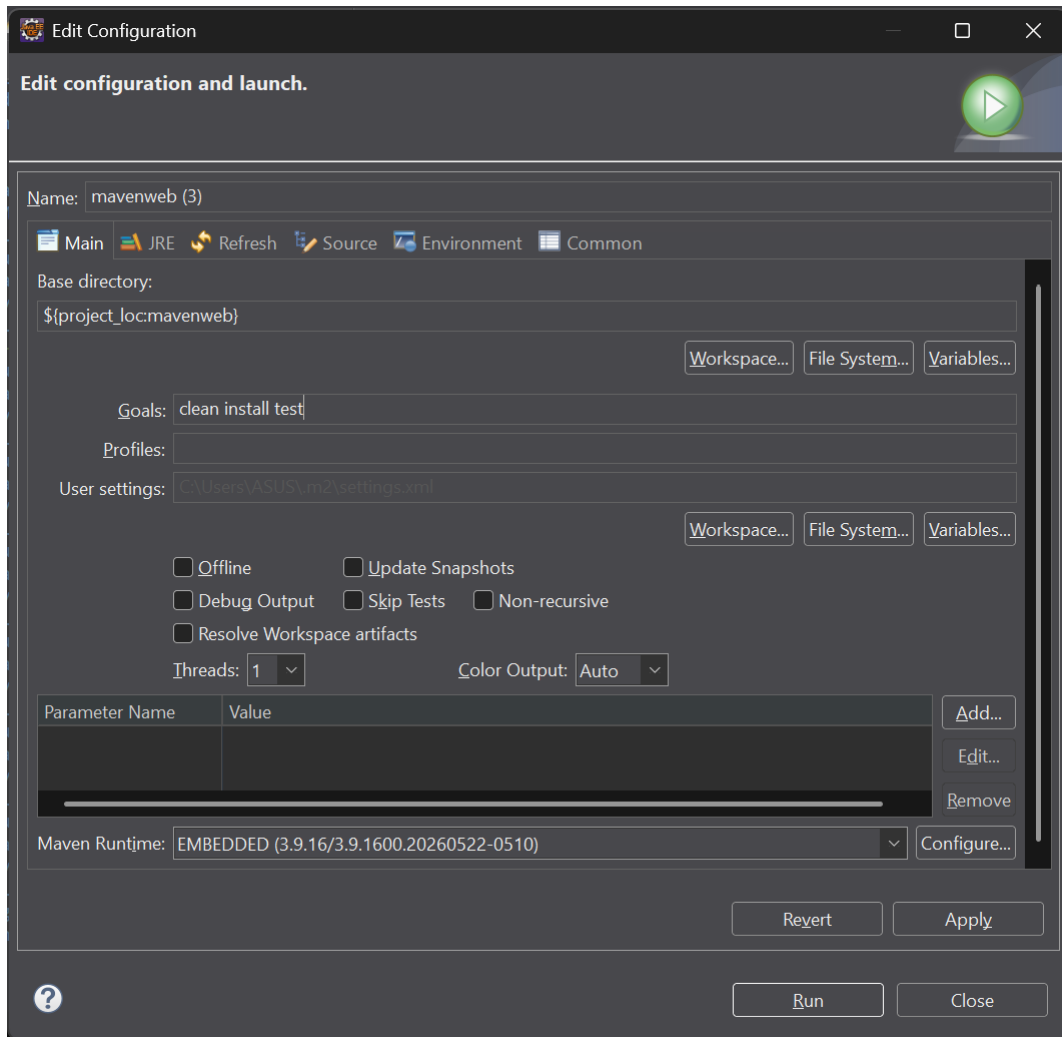
└─ Right-click on index.jsp -> Run As -> Maven Build



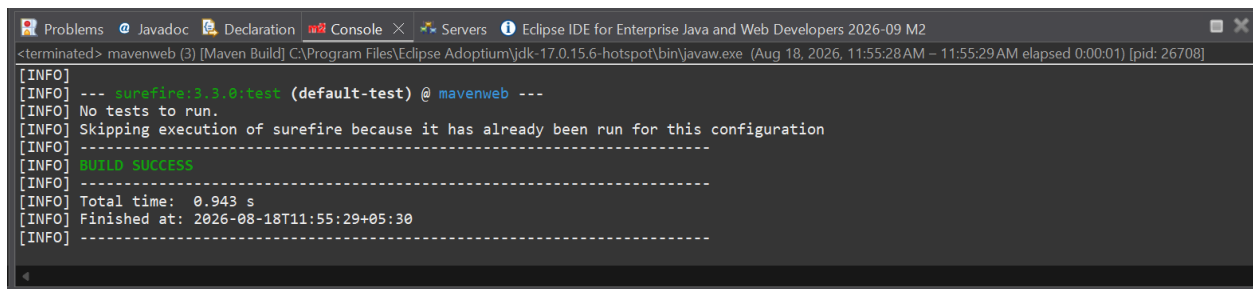
Step 9. In the Maven Build dialog:

└─ Enter Goals: clean install test

└─ Click on Apply -> Click on Run



Step 10. Check console for BUILD SUCCESS message.



Step 11. Run the application:

- └─ Right-click on index.jsp -> Run As -> Run on Server
- └─ Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.



Note:-Now push yours Maven java project and Maven Web Project into your github

Maven Java Project

```
ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ git init
Reinitialized existing Git repository in C:/Users/ASUS/eclipse-workspace/Simple_Calculator/.git/

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ echo -e "target/\n.settings/\n.classpath\n.project" > .gitignore

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ git commit -m "Initial commit: Added Maven Calculator Project"
[main (root-commit) ac4f160] Initial commit: Added Maven Calculator Project
 4 files changed, 98 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 pom.xml
 create mode 100644 src/main/java/Calculator.java
 create mode 100644 src/main/java/Main.java

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ git remote add origin https://github.com/DiamondDevil-dotcom/maven-calculator.git

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 32 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 1.58 KiB | 1.58 MiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DiamondDevil-dotcom/maven-calculator.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$
```

Maven Web Project

```
MINGW64:/c/Users/ASUS/eclipse-workspace/mavenweb

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/mavenweb (main)
$ git init
Reinitialized existing Git repository in C:/Users/ASUS/eclipse-workspace/mavenweb/.git/

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/mavenweb (main)
$ git add .

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/mavenweb (main)
$ git commit -m "maven web push"
On branch main
nothing to commit, working tree clean

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/mavenweb (main)
$ git remote add origin https://github.com/DiamondDevil-dotcom/mavenweb.git

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/mavenweb (main)
$ git push -u origin main
Enumerating objects: 35, done.
Counting objects: 100% (35/35), done.
Delta compression using up to 32 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (35/35), 6.07 KiB | 1.52 MiB/s, done.
Total 35 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/DiamondDevil-dotcom/mavenweb.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/mavenweb (main)
$ |
```

3. Configure Java Version via Compiler Plugin

In `pom.xml`:

```
<plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-compiler-plugin</artifactId>

  <version>3.11.0</version>

  <configuration>

    <source>17</source>

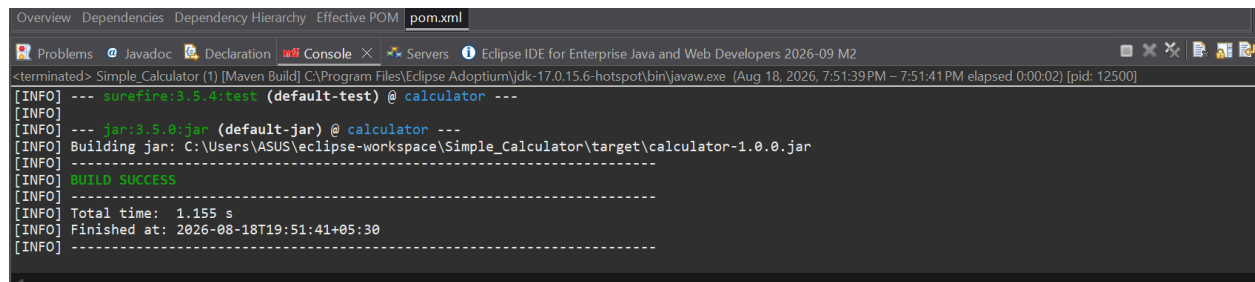
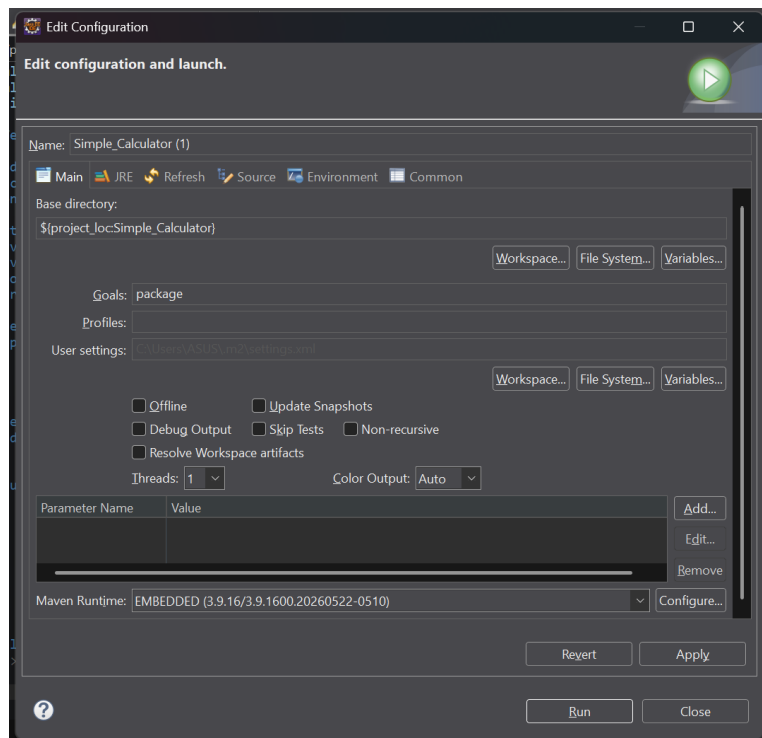
    <target>17</target>

  </configuration> </plugin>
```

```
Simple_Calculator/pom.xml ×
10 <project xmlns="http://maven.apache.org/POM/4.0.0"
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>com.yourname</groupId>
8     <artifactId>calculator</artifactId>
9     <version>1.0.0</version>
10
11 <properties>
12     <maven.compiler.source>17</maven.compiler.source>
13     <maven.compiler.target>17</maven.compiler.target>
14     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15 </properties>
16
17 <dependencies>
18 <dependency>
19     <groupId>org.junit.jupiter</groupId>
20     <artifactId>junit-jupiter-api</artifactId>
21     <version>5.10.0</version>
22     <scope>test</scope>
23 </dependency>
24 </dependencies>
25
26 <build>
27 <plugins>
28 <plugin>
29     <groupId>org.apache.maven.plugins</groupId>
30     <artifactId>maven-compiler-plugin</artifactId>
31     <version>3.11.0</version>
32 <configuration>
33     <source>17</source>
34     <target>17</target>
35 </configuration>
36 </plugin>
37 </plugins>
38 </build>
39
40 </project>
```

Check:

- Run `mvn package`
- Inspect generated JAR: `jar tf target/myapp.jar`



4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```
public class AppTest {

    @Test

    public void testSum() {

        assertEquals(5, 2 + 3);

    }

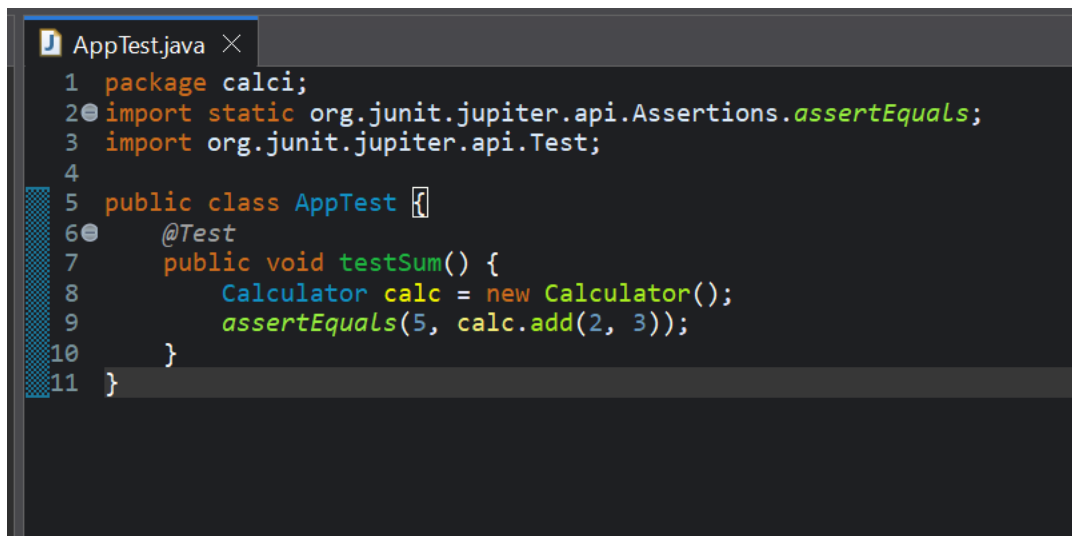
}
```

Run:

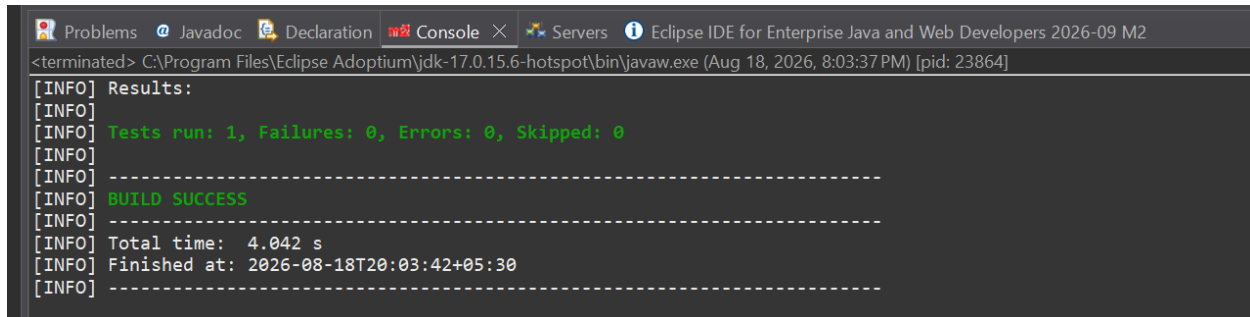
mvn test

Check:

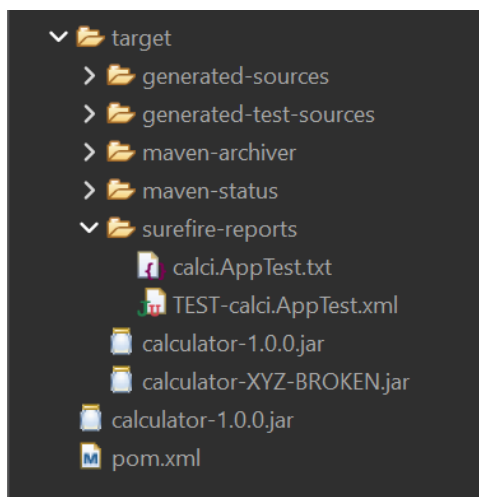
- target/test-classes/ → compiled test .class files
 - target/surefire-reports/ → .txt or .xml test results
- If test fails → corresponding log and failure trace shown



```
AppTest.java ×
1 package calci;
2 import static org.junit.jupiter.api.Assertions.assertEquals;
3 import org.junit.jupiter.api.Test;
4
5 public class AppTest {
6     @Test
7     public void testSum() {
8         Calculator calc = new Calculator();
9         assertEquals(5, calc.add(2, 3));
10    }
11 }
```



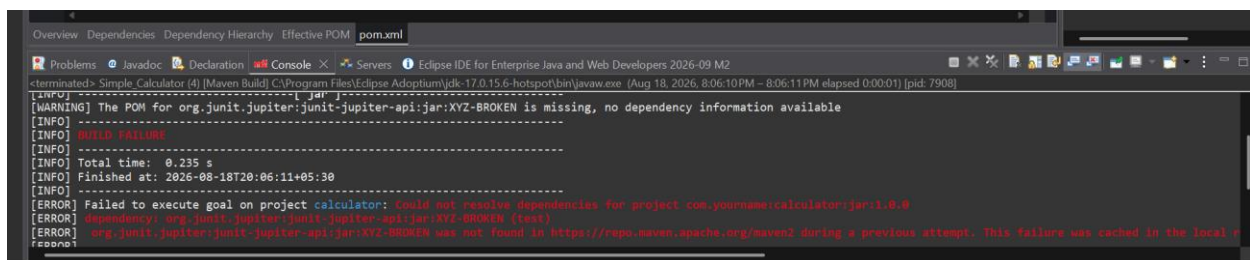
```
Problems Javadoc Declaration Console Servers Eclipse IDE for Enterprise Java and Web Developers 2026-09 M2
<terminated> C:\Program Files\Eclipse Adoptium\jdk-17.0.15.6-hotspot\bin\javaw.exe (Aug 18, 2026, 8:03:37 PM) [pid: 23864]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.042 s
[INFO] Finished at: 2026-08-18T20:03:42+05:30
[INFO] -----
```



5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause BUILD FAILURE
- Maven shows precise error in console
- Fix the dependency tag → re-run mvn clean install

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
10 <project xmlns="http://maven.apache.org/POM/4.0.0"
11     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
12     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
13
14     <modelVersion>4.0.0</modelVersion>
15
16     <groupId>com.yourname</groupId>
17     <artifactId>calculator</artifactId>
18     <version>1.0.0</version>
19
20     <properties>
21         <maven.compiler.source>17</maven.compiler.source>
22         <maven.compiler.target>17</maven.compiler.target>
23         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
24     </properties>
25
26     <dependencies>
27         <dependency>
28             <groupId>org.junit.jupiter</groupId>
29             <artifactId>junit-jupiter-api</artifactId>
30             <version>XYZ-BROKEN</version>
31             <scope>test</scope>
32         </dependency>
33     </dependencies>
34
35     <build>
36         <plugins>
37             <plugin>
38                 <groupId>org.apache.maven.plugins</groupId>
39                 <artifactId>maven-compiler-plugin</artifactId>
40                 <version>3.11.0</version>
41                 <configuration>
42                     <source>17</source>
43                     <target>17</target>
44                 </configuration>
45             </plugin>
46         </plugins>
47     </build>
48 </project>
```



The screenshot shows the Eclipse IDE console with the following output:

```
<terminated> Simple_Calculator (4) [Maven Build] C:\Program Files\Eclipse Adoptium\jdk-17.0.15-hotspot\bin\javaw.exe (Aug 18, 2026, 8:06:10 PM - 8:06:11 PM elapsed: 0:00:01) [pid: 7908]
[INFO] -----
[WARNING] The POM for org.junit.jupiter:junit-jupiter-api:jar:XYZ-BROKEN is missing, no dependency information available
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 0.235 s
[INFO] Finished at: 2026-08-18T20:06:11+05:30
[INFO] -----
[ERROR] Failed to execute goal on project calculator: Could not resolve dependencies for project com.yourname:calculator:jar:1.0.0
[ERROR] dependency: org.junit.jupiter:junit-jupiter-api:jar:XYZ-BROKEN (test)
[ERROR] org.junit.jupiter:junit-jupiter-api:jar:XYZ-BROKEN was not found in https://repo.maven.apache.org/maven2 during a previous attempt. This failure was cached in the local r
[ERROR]
```

6. Adding Resource Files

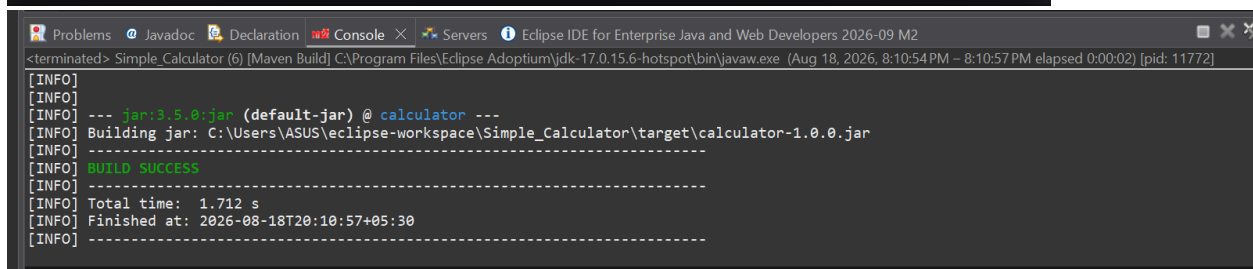
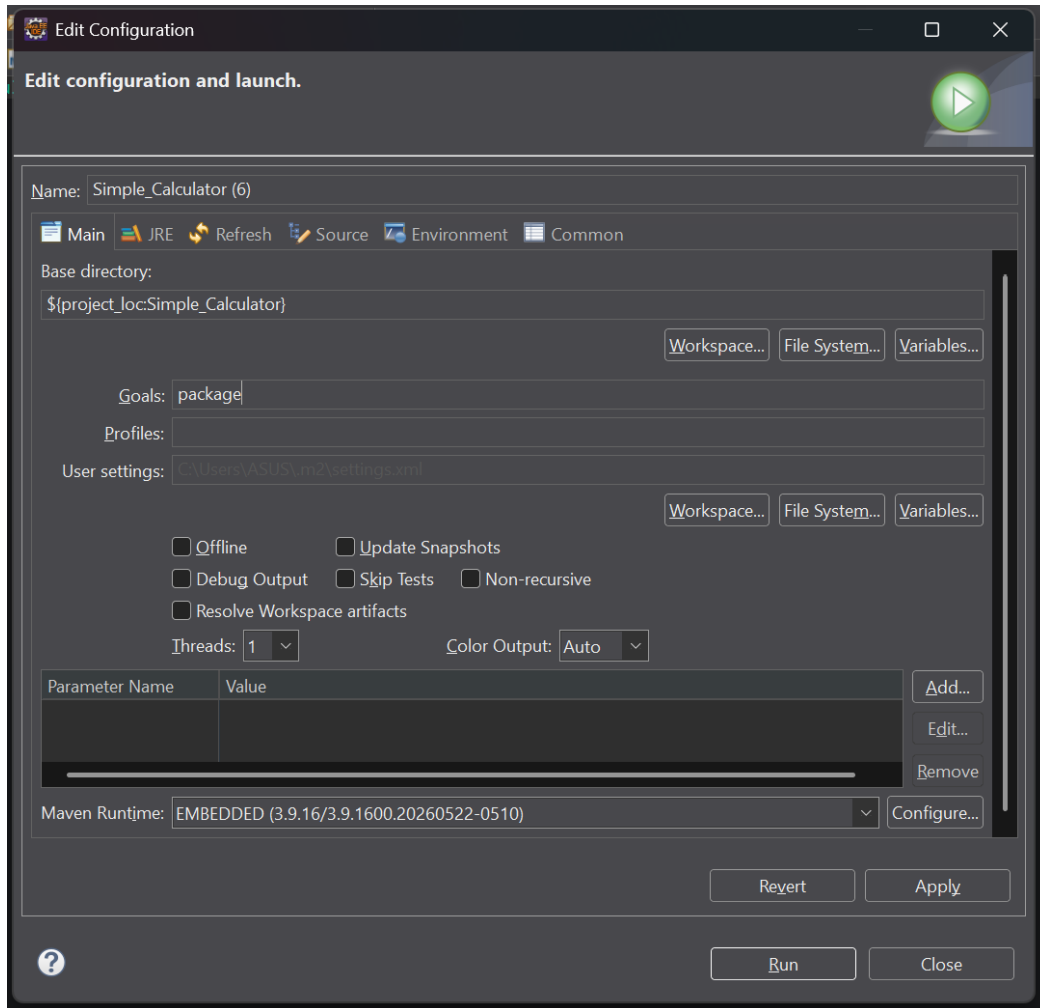
Put config.properties inside:

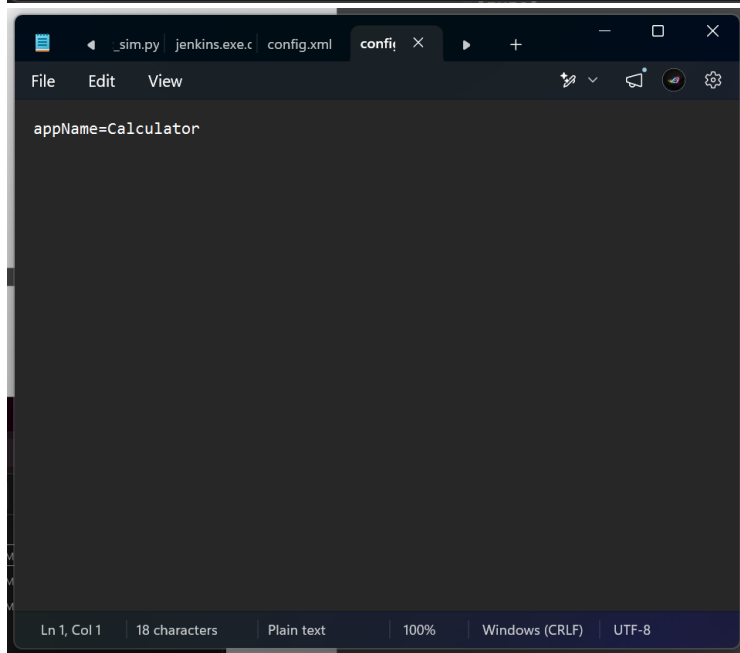
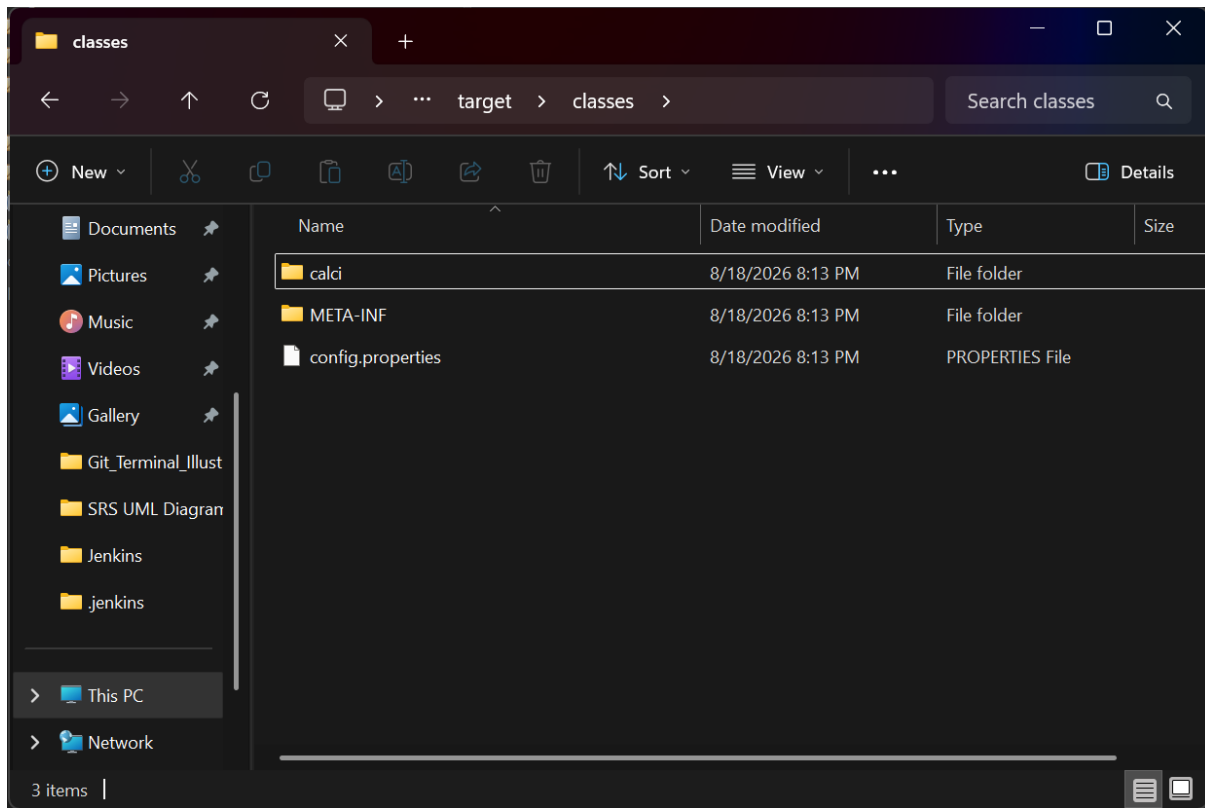
src/main/resources/config.properties

To read:


```
InputStream input =  
getClass().getClassLoader().getResourceAsStream("config.properties");
```

After build, check target/classes/ → file should exist there.





7. Multi-Module and Parent Projects

Structure:

```

parent/
├── pom.xml (packaging: pom)
├── core/
│   └── pom.xml

```

└─ web/
└─ pom.xml

- Parent pom.xml defines <modules> and common dependencies
- Each submodule builds independently into its target/ directory

New Maven Project

New Maven project

Configure project

Artifact

Group Id: com.lab

Artifact Id: parent-project

Version: 0.0.1-SNAPSHOT

Packaging: pom

Name:

Description:

Parent Project

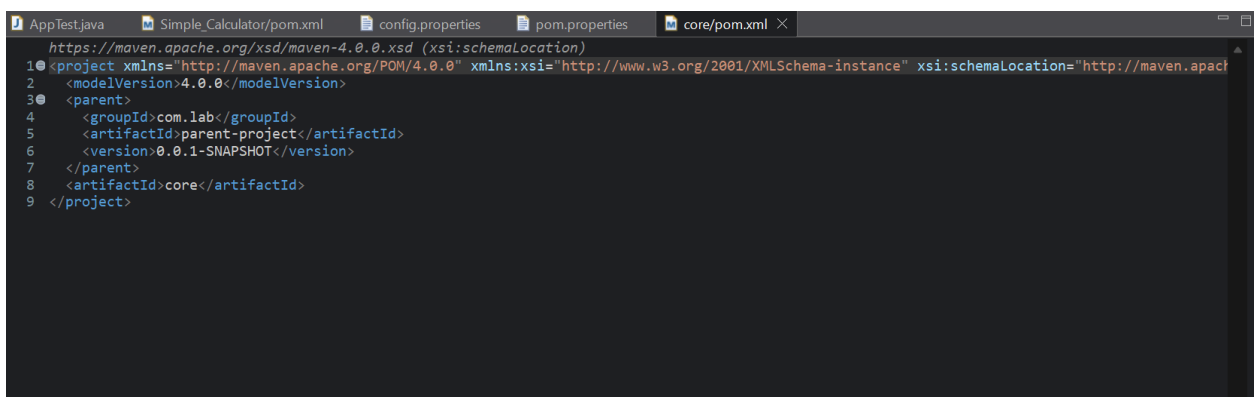
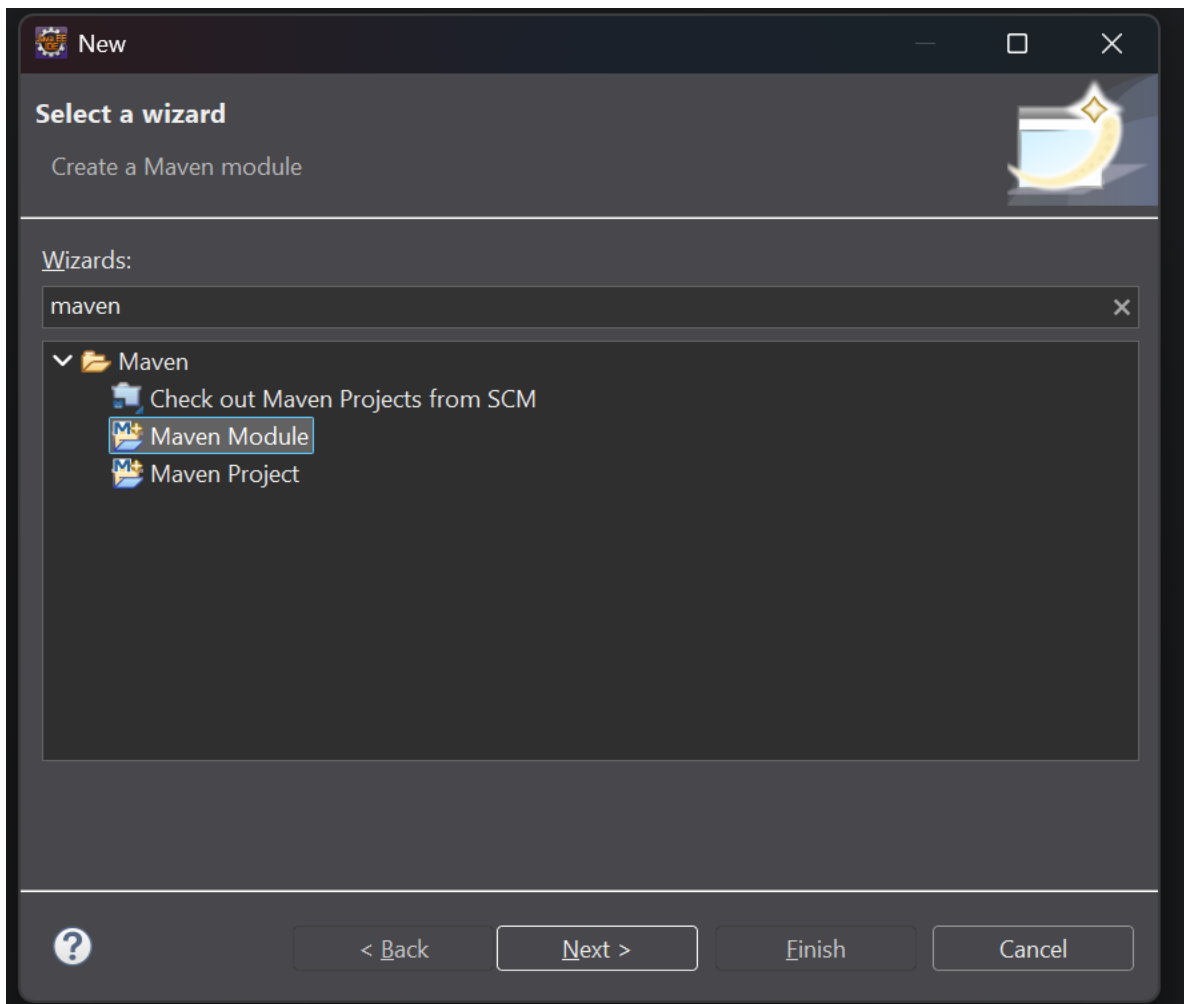
Group Id:

Artifact Id:

Version: Browse... Clear

▶ Advanced

? < Back Next > Finish Cancel



8. Executable JARs

Add to pom.xml:

```
<build>
  <plugins>
```

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <configuration>
    <archive>
      <manifest>
        <mainClass>com.example.Main</mainClass>
      </manifest>
    </archive>
  </configuration>
</plugin>
</plugins>
</build>

```

Run:

```

mvn package
java -jar target/myapp.jar

```

```

MINGW64:/c/Users/ASUS/eclipse-workspace/Simple_Calculator

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$ java -jar target/calculator-1.0.0.jar
=== Maven Calculator Started ===
Enter first number: 10
Enter operation (+, -, *, /): +
Enter second number: 20
Result: 30.0
Calculate again? (y/n): n
-----
Calculator closed.

ASUS@Rog-Strix MINGW64 ~/eclipse-workspace/Simple_Calculator (main)
$

```

9. Building a WAR File

Create a Maven web project with structure:

```

src/main/webapp/
└─ WEB-INF/web.xml

```

Add:

```

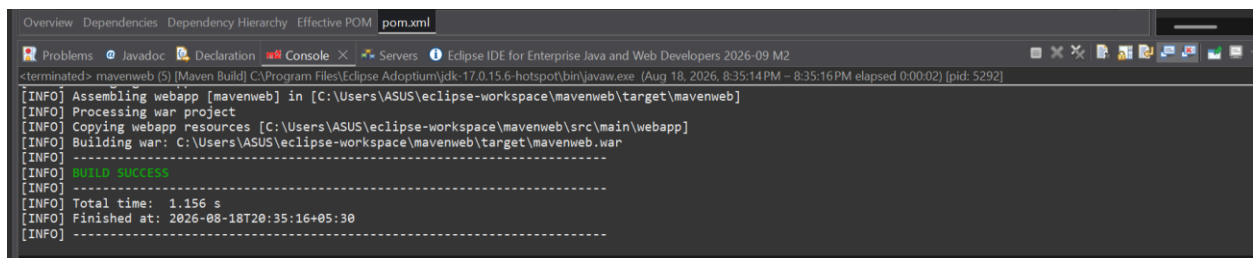
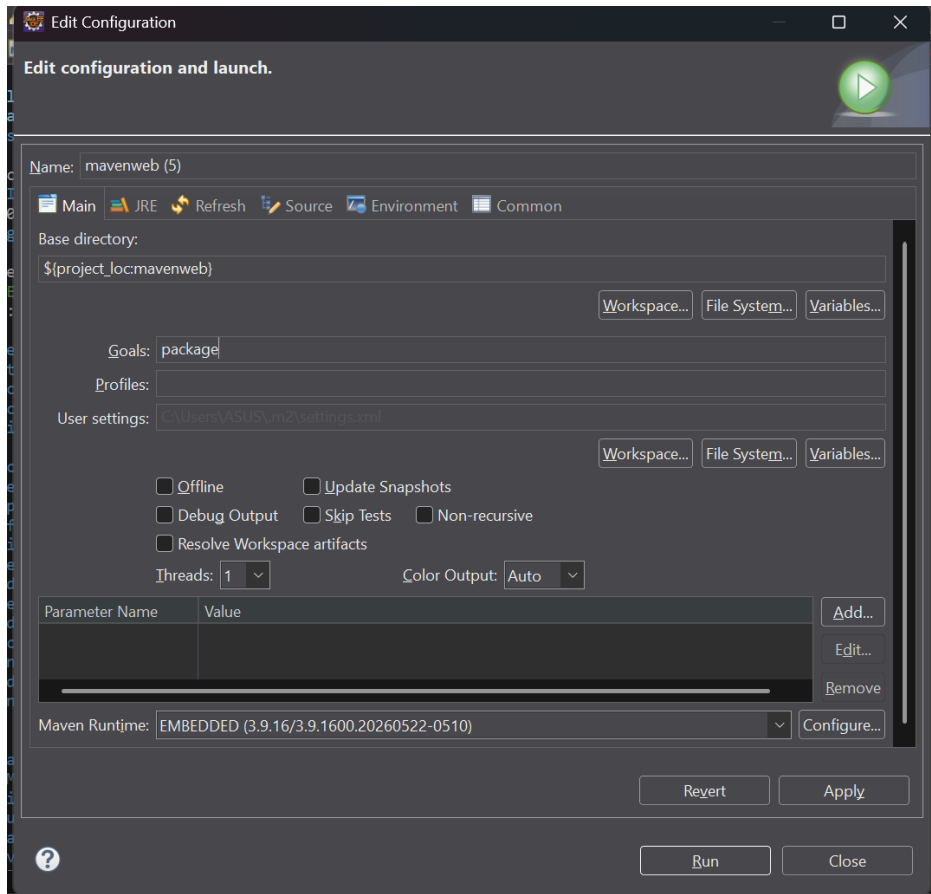
<packaging>war</packaging>

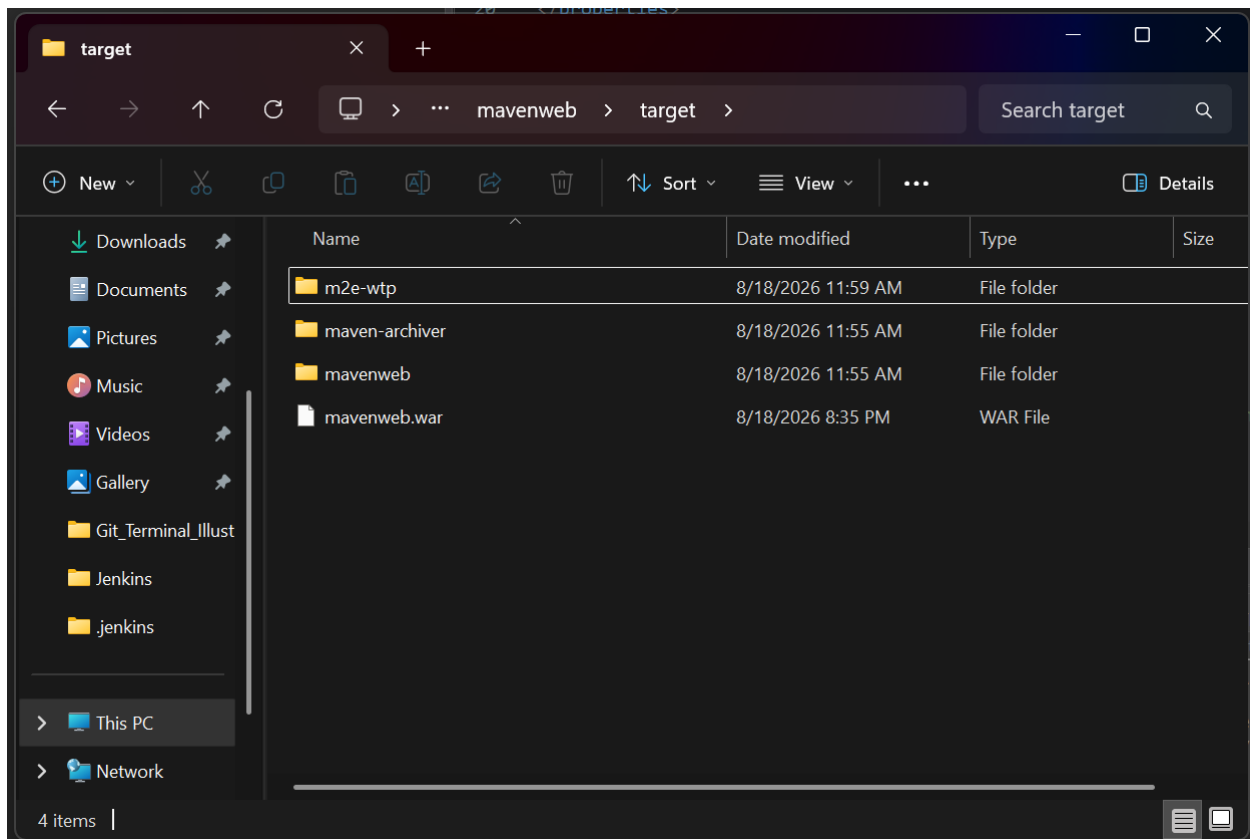
```

Command:

`mvn package`

Generates target/mywebapp.war → deploy on Tomcat server.





10. Scenario-Based Questions:

1. If my error is about:

Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.8.1:compile [ERROR] -> [Help 1]

[ERROR] To see the full stack trace of the errors, re-run Maven with the -X switch.

How can I resolve it using maven terminal?

Command: Run `mvn clean compile -e` (for error stack trace) or `mvn clean compile -X` (for full debug logging).

Resolution Steps: Check the log output for syntax errors, missing dependencies, or mismatches between your project's configured Java source/target version and the actual JDK installed on your machine.

2. A dependency you added is not recognized by the compiler. What steps would you take to confirm it is available in `.m2` and listed in dependency tree?

Dependency Tree: Run `mvn dependency:tree` in the terminal to verify that Maven successfully resolved and included the artifact in the project hierarchy.

Local Repository (.m2): Check your local .m2 directory (`~/.m2/repository/` or `C:\Users\<User>\.m2\repository\`) under the matching `groupId/artifactId/version` path to ensure the `.jar` and `.pom` files were downloaded.

3. A teammate sends a `.patch` file for a bug fix. How would you apply it and include it in your Maven build?

Apply the Patch: Run `git apply <filename>.patch` (or `patch -p1 < <filename>.patch` on Unix systems) from the project root.

Rebuild: Execute `mvn clean install` to compile the changes, run tests, and package the updated build.

4. You have multiple **JUnit** test failures and want to rerun only failed tests. How would you approach this?

Specific Execution: Run `mvn test -Dtest=TestClassName` (to run a specific class) or `mvn test -Dtest=TestClassName#failedMethodName` (to target a specific test method).

Automatic Retries: If using Maven Surefire 2.19.1+, pass `-Dsurefire.rerunFailingTestsCount=2` to automatically retry failed tests during the build.

5. Your Maven build fails due to “Unsupported class version error.” What plugin and configuration would you review?

Plugin & Configuration: Review the `maven-compiler-plugin` inside `pom.xml`.

Resolution: Check the `<source>` and `<target>` (or `<release>`) tags to ensure the bytecode version generated matches or is compatible with the JRE running the application.

6. You need to change your Java application from a **WAR** to a standalone **JAR**. What `pom.xml` changes are needed?

pom.xml Packaging: Change `<packaging>war</packaging>` to `<packaging>jar</packaging>`.

Plugin Setup: Add a plugin like `maven-shade-plugin` or `maven-assembly-plugin` (or `spring-boot-maven-plugin` for Spring Boot) to bundle dependencies into an executable "fat JAR" and specify the main entry-point class.

7. You are required to change the default build output directory from `target/` to `build_output/`. How would you configure it?

Configure the `<build>` section of your `pom.xml`:

```
<build> <directory>build_output</directory> </build>
```

8. You want to skip tests during the Maven build. What command would you use?

Compile tests but do not run them: `mvn clean install -DskipTests`

Skip test compilation and execution entirely: `mvn clean install -Dmaven.test.skip=true`

9. How would you generate a site report (with test coverage, dependency analysis) for a Maven project?

Command: Run `mvn site`.

Configuration: Add reporting plugins like `jacoco-maven-plugin` (for code coverage) and `maven-project-info-reports-plugin` (for dependency analysis) in the `pom.xml`. The generated site report will be saved to `target/site/`.

10. How do you build a Java project using Maven, and what files are generated in the `target/` folder after running `mvn clean install`?

`target/classes/`: Compiled application `.class` files.

`target/test-classes/`: Compiled test `.class` files.

`target/surefire-reports/`: XML/TXT test execution reports.

`target/<artifact-name>.<jar/war>`: The final packaged binary file.

11. How does Maven resolve dependency conflicts when two libraries use different versions of the same dependency, and how can you view and manage the **dependency tree**?

Conflict Resolution: Maven uses the "nearest definition" rule (the version closest to the root in the dependency tree takes precedence).

Viewing: Run `mvn dependency:tree` to inspect transitive dependencies.

Managing: Enforce specific versions using `<dependencyManagement>` or exclude specific transitive libraries using `<exclusions>` inside individual `<dependency>` blocks.

12. How do you write and run a JUnit test in a Maven project, and where are the compiled test classes and reports stored after running **mvn test**?

Location & Execution: Place test files under `src/test/java/` and execute them using `mvn test`.

Output Paths: Compiled test classes go to `target/test-classes/` and generated test reports go to `target/surefire-reports/`.

13. How can you create an executable **JAR** with a main method using Maven, and which **plugin** helps configure this behavior?

Plugin: Use the `maven-shade-plugin` or `maven-assembly-plugin`.

Configuration: Configure the plugin in `pom.xml` to generate a manifest containing the `<mainClass>` entry pointing to your main class.

14. How do you install and use a custom third-party JAR file in your Maven project, and how can you confirm it's included in the build and **classpath**?

Install Command:

```
mvn install:install-file -Dfile=/path/to/library.jar -DgroupId=com.custom -DartifactId=custom-lib -Dversion=1.0 -Dpackaging=jar
```

Usage: Add the corresponding `groupId`, `artifactId`, and `version` to your `pom.xml` `<dependencies>` section, then verify inclusion via `mvn dependency:tree`.

15. How do you create a Maven web project that packages into a WAR file, and what is the standard folder structure for such a project?

`src/main/java/` — Java source code and servlets.

`src/main/resources/` — Application properties and configuration files.

`src/main/webapp/` — Web assets (HTML, JSP, CSS, JS).

src/main/webapp/WEB-INF/ — Web configuration including `web.xml`.

16. What command do you use to build a WAR file in Maven, where is it generated, and how can you deploy it to a server like **Apache Tomcat**?

Build Command: `mvn clean package`.

Location: Generated in the `target/` directory.

Deployment: Copy the `.war` file to the server's deployment directory (e.g., Tomcat's `webapps/` folder) or automate deployment via Jenkins ("Deploy to container" plugin) / Tomcat Maven Plugin.

17. How do you add JSTL and **servlet-api dependencies** in a Maven web project, and why should the servlet API use provided scope instead of compile?

Configuration: Declare both under `<dependencies>` in `pom.xml`.

Why provided scope: The servlet container (e.g., Tomcat) provides its own Servlet API implementation at runtime. Using `<scope>provided</scope>` ensures the JAR is used only for compilation and is not bundled into the final WAR, avoiding runtime class-loader conflicts (`ClassCastException`).

18. How do you set up a multi-module Maven web project with separate modules for core logic and web interface, and how are these modules built and connected?

Structure: A root parent `pom.xml` with `<packaging>pom</packaging>` listing `<modules>` (e.g., `<module>core</module>`, `<module>web</module>`).

Connection: The `web` module declares the `core` module as a dependency in its `pom.xml`.

Build Order: Running `mvn clean install` at the root compiles modules in topological dependency order (`core` first, then `web`).

19. How do you configure a Maven web project, and how does its packaging and execution differ from a traditional WAR-based application?

Traditional WAR: Requires an external, standalone servlet container (like Tomcat) installed and running to host and unpack the WAR file.

Modern Approach (e.g., Spring Boot): Packages an embedded web server directly inside a standalone JAR. The application can run independently on any system with Java installed via `java -jar application.jar`, without configuring external server infrastructure.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.